# COSMOS ][ - 実装仕様

| | |
|---|---|
| アーカイブ | ☐ |
| 編集日 | @2026年3月3日 7:52 |

# 実装仕様 — 対話型インタプリタ実行環境（COSMOS II）【32bit符号付整数版】

以下の実装仕様に基づいて「対話型インタプリタ実行環境（COSMOS II）」のインタプリタ部分を構築する。
本仕様はプログラムコードの実行制御部分に関する内部設計・実装ガイドラインを対象とする。

---

## 1. 実装ガイド

### 1.1 モジュール構成

| モジュール | 役割 |
|---|---|
| **Tokenizer** | ソースコードをトークンに分解する。`%` の曖昧性解決（言語仕様 §2.2）もここで行う。 |
| **Parser** | トークン列を解析し、文の種類（代入、制御、ループ、入出力等）を判別する。構文エラーを検出する。**演算子の優先順位に基づいた再帰下降パーサ**を実装する。 |
| **Evaluator** | 言語仕様の優先順位規則に基づき式を評価する。括弧の再帰的処理と単項演算子の即時結合を処理する。すべての演算は符号付32bitで行う。 |
| **Executor** | PC（行番号）と StatementIndex（文番号）の2階層ポインタで実行を制御する。ジャンプ、サブルーチン、ループの遷移を管理する。 |

### 1.2 実行ループのアルゴリズム

1. **行の特定**: PC が指す行番号を取得する。
2. **文の分割**: 行内の文をトークンリストとして取得する。
3. **逐次実行**:
    - StatementIndex が指す文を1つ実行する。
    - IF文（`;=`）が偽なら、行の残りの文を破棄してステップ1へ（PCを次の行に進める）。
    - GOTO/GOSUB なら PC を書き換え、ステップ1へ。
    - RETURN なら CallStack から復帰先を取得し、その位置から再開。
    - ループ制御（`@`, `@=`）なら LoopStack を操作し、条件に応じてジャンプ。
    - 上記以外なら StatementIndex をインクリメントし、次の文へ。
4. **中断のフック**: 文の実行の合間に、キーボードからの強制中断（`CTRL+C`）が入力されていないか判定する。中断フラグが立っている場合はエラー扱いでプログラムを停止する。

5. **終了判定**: PC が `1`(END) に達するか、プログラム末尾を超えたら終了。

> [!NOTE]
> 変数（ユーザー変数・`%`）の初期化は Memory バッファ生成時に行う（言語仕様 §1.2 参照）。

## 1.3 実行コンテキスト（Context）

実行時に保持すべきデータ構造:

| データ | 型 | 説明 |
|---|---|---|
| `Memory` | `uint8[65536]` | 64KBのバイト配列。変数・システム変数・フリー領域・プログラム領域を含む。アドレスは常に無符号。 |
| `PC` | `int32` | 現在実行中の行番号。`-1` はプログラム終了を示す。 |
| `StatementIndex` | `int32` | 行内での文番号（0始まり）。 |
| `CallStack` | `Stack<(PC, StatementIndex)>` | GOSUB の戻り先を保持する LIFO スタック。 |
| `LoopStack` | `Stack<(PC, StatementIndex)>` | DO(`@`) / FOR の開始位置を保持するスタック。 |

## 1.4 メモリモデル（メモリマップとバイトオーダー）

インタープリタが管理する仮想メモリは **64KB**（`$0000`〜`$FFFF`）とし、以下のレイアウトで物理的に実装する。

| アドレス範囲 | 領域名 | 用途 |
|---|---|---|
| `$0000`〜`$00CF` | 変数領域 | ユーザー変数 A-Z, a-z を各4バイトで連続配置 |
| `$00D0`〜`$00D3` | システム変数領域 | システム変数 `%`（除算の剰余）を4バイト（big-endian）で格納 |
| `$00D4`〜`$00FF` | 予約領域 | 将来のシステム拡張用 |
| `$0100`〜`$7FFF` | フリー領域 | 配列参照で利用可能なユーザー自由領域 |
| `$8000`〜`$FFFF` | プログラム領域 | プログラムソースコード |

インタープリタ実行用スタック（CallStack, LoopStack）等は、上記メモリマップとは独立した領域で管理する。

### 変数アドレスの配置

```
A = $0000, B = $0004, ..., Z = $0064
a = $0068, b = $006C, ..., z = $00CC
```

### バイトオーダーと配列アクセスのルール

変数の値はメモリ上において**ビッグエンディアン**で保持・エミュレートする。

`A = $12345678`（10進: 305419896）のとき:

| 参照 | アドレス | 値 |
|---|---|---|
| バイト参照 `A[0]` | `$0000` | `$12`（最上位バイト） |
| バイト参照 `A[1]` | `$0001` | `$34` |
| バイト参照 `A[2]` | `$0002` | `$56` |

| 参照 | アドレス | 値 |
|---|---|---|
| バイト参照 `A[3]` | `$0003` | `$78` （最下位バイト） |
| ワード参照 `A:0]` | `$0000` - `$0001` | `$1234` （符号付16bit: 4660） |
| ワード参照 `A:1]` | `$0002` - `$0003` | `$5678` （符号付16bit: 22136） |
| ダブルワード参照 `A{0}` | `$0000` - `$0003` | `$12345678` （符号付32bit: 305419896） |
| ダブルワード参照 `A{1}` | `$0004` - `$0007` | 隣接変数Bの値 |

`A = -1` （内部表現: `$FFFFFFFF` ）のとき:

| 参照 | アドレス | 値 |
|---|---|---|
| バイト参照 `A[0]` | `$0000` | `$FF` |
| バイト参照 `A[1]` | `$0001` | `$FF` |
| バイト参照 `A[2]` | `$0002` | `$FF` |
| バイト参照 `A[3]` | `$0003` | `$FF` |
| ワード参照 `A:0]` | `$0000` - `$0001` | `$FFFF` （符号付16bit: -1） |
| ダブルワード参照 `A{0}` | `$0000` - `$0003` | `$FFFFFFFF` （符号付32bit: -1） |

> [!WARNING]
> 実行環境（x86/x64、JSのTypedArray等）はリトルエンディアンが多いため、`Memory[]` 配列（Uint8Array等）への読み書き時に**明示的なバイトスワップ（エンディアン変換）が必要**である。

> [!NOTE]
> 各配列参照の読み出し型（言語仕様 §4.2 参照）の解釈は Evaluator にて処理する。

---

# 2. エラー仕様

## 2.1 構文エラー（SYNTAX ERROR）

字句解析・構文解析フェーズで不正な記述を検出した場合、実行を開始せずにエラーを出力する。

| 要因 | エラーメッセージ |
|---|---|
| 未定義の記号 | `** SYNTAX ERROR : Undefined Symbol` |
| 閉じられていない括弧 `( )`, `{ }` | `** SYNTAX ERROR : Unclosed bracket` |
| システム変数（`%`, `'`）への代入試行 | `** SYNTAX ERROR : Overriding a system variable` |
| 行頭以外での行番号記述 | `** SYNTAX ERROR : Invalid location for line no.` |
| その他の構文違反 | `** SYNTAX ERROR : Undefined syntax` |

## 2.2 実行時エラー（RUNTIME ERROR）

プログラム実行中に以下の状態が発生した場合、直ちに実行を停止し、エラーメッセージを表示してエディタに戻る。

| 要因 | エラーメッセージ |
|---|---|
| `/` の右辺が `0` | `** RUNTIME ERROR : Division by zero` |
| 配列参照のアドレスが 0〜65535 範囲外 | `** RUNTIME ERROR : Address out of range` |
| CallStack が空の状態で `]` を実行 | `** RUNTIME ERROR : Stack underflow` |

| 要因 | エラーメッセージ |
|---|---|
| `!` （GOSUB）のネストが実装上限を超過 | `** RUNTIME ERROR : Stack overflow` |
| 対応する `@` なしで `@=(式)` / `@=値` を実行 | `** RUNTIME ERROR : Mismatched control blocks` |
| GOTO/GOSUB 先の行が存在せず、より大きい行番号もない | `** RUNTIME ERROR : Undefined line number` |
| その他予期しないエラー | `** RUNTIME ERROR : Unexpected error` |

### 2.2.1 強制中断（INTERRUPT）

`CTRL+C` キー入力によるユーザー操作での強制中断は、RUNTIME ERROR とは内部的に区別される独立したカテゴリ（`INTERRUPT`）として扱う。

| 要因 | エラーメッセージ |
|---|---|
| ユーザーによる `CTRL+C` キー入力での強制中断 | `** Program interrupted` |

> [!NOTE]
> 表示形式は RUNTIME ERROR と同一だが、エラーの発生起因として区別するため内部カテゴリを分離する。

## 2.3 入力エラー

- **数値入力** `?`: 数値以外の入力、または 32bit 符号付整数の範囲外（-2147483648〜2147483647 以外）の値が入力された場合は、エラーとせず、再入力を促すプロンプトを表示して待機する。

## 2.4 エラー表示形式

すべてのエラーは**エラー表示前に2行の空行**を出力し、以下の形式で表示する。

```
<空行>
<空行>
<エラーメッセージ>
<エラーが発生した行の内容>
<エラー箇所の列位置に "^" を表示>
```

- RUNTIME ERROR および INTERRUPT ERROR のエラー箇所は、エラーが発生した**文の先頭文字**の位置とする。
- `^` の列位置は `<行番号> <ソース>` の形式で表示された行内における文字位置とする（行番号とスペース1文字の分だけオフセットする）。

例（SYNTAX ERROR）：

```
** SYNTAX ERROR : Undefined Symbol
10 A = B + @
           ^
```

例（RUNTIME ERROR）：

```
** RUNTIME ERROR : Division by zero
```

```
10 A = B / 0
           ^
```

例（INTERRUPT）：

```
** Program interrupted
10 "WARAWA "
^
```